

شرح قاعدة بيانات نظام

إدارة المخزون والمبيعات

Prisma Schema - Multi-Tenant Inventory, Sales & Analytics

شرح منظم لكل جدول، كل حقل، العلاقات، والهدف من التصميم

الجدول: Account - User - Category - Product - StockMovement - Expense - Sale - DailyReport

نظرة عامة على بنية النظام

الفكرة الأساسية: كل محل داخل النظام يمثل Account مستقل. المستخدمون والمنتجات والمبيعات والمصاريف والتقارير ترتبط بهذا الحساب، وبذلك تبقى بيانات كل محل منفصلة عن المحلات الأخرى.

#	الجدول	النوع	الدور
1	Account	جذر النظام	يمثل كل محل أو Business، وكل البيانات الأخرى ترجع إليه.
2	User	حالة وصلاحيات	المستخدمون: OWNER و SUPER_ADMIN و WAREHOUSE.
3	Category	حالة	تصنيف المنتجات داخل كل محل.
4	Product	حالة حالية	المنتج الحالي: الكلفة، الهامش، المخزون، والسعر المقترح.
5	StockMovement	سجل تاريخي	كل حركة إدخال مخزون مع الكمية والكلفة وقتها.
6	Expense	سجل تاريخي	المصاريف العامة للمحل.
7	Sale	مصدر الحقيقة للمبيعات	كل عملية بيع مع snapshot للكلفة والفئة وقت البيع.
8	DailyReport	ملخص / Cache	نتيجة يومية محسوبة مسبقا لتسريع Dashboard والتقارير.

دليل شرح قاعدة البيانات - Inventory & Sales SaaS

تسلسل البيانات بشكل مبسط

```
SUPER_ADMIN
|
Platform
|
Account
|-- Users
|-- Categories --> Products --> StockMovement
|                               \-> Sales
|-- Expenses                  |
\-----> DailyReport
```

الفكرة التي يجب حفظها

Product يخبرك بوضع المنتج الآن. StockMovement يخبرك كيف تغير المخزون عبر الزمن. Sale يخبرك ماذا بيع فعليا. Expense يخبرك ماذا دفع المحل. DailyReport يعطيك ملخصا سريعا لنتيجة اليوم.

الكامل Prisma Schema

هذا هو الشكل الكامل الذي تقوم الأقسام التالية بشرحه بالتفصيل:

```
enum UserRole {
  SUPER_ADMIN
  OWNER
  WAREHOUSE
}

model User {
  id          String    @id @default(cuid())
  email       String    @unique
  password    String
  role        UserRole
  accountId   String?
  account     Account? @relation(fields: [accountId], references: [id])
  createdAt   DateTime @default(now())
}

model Account {
  id          String    @id @default(cuid())
  name        String
  isActive    Boolean   @default(true)
  users       User[]
  categories  Category[]
  products    Product[]
  expenses    Expense[]
  sales       Sale[]
  reports     DailyReport[]
  createdAt   DateTime @default(now())
}

model Category {
  id          String    @id @default(cuid())
  accountId   String
  account     Account @relation(fields: [accountId], references: [id])
  name        String
  products    Product[]
  createdAt   DateTime @default(now())

  @@index([accountId])
}

model Product {
  id          String    @id @default(cuid())
  accountId   String
  account     Account @relation(fields: [accountId], references: [id])
  categoryId  String
  category    Category @relation(fields: [categoryId], references: [id])
  name        String
  imageUrl    String
  color       String
  wholesaleCost Float?
  profitMargin Float?
  currentStock Int       @default(0)
  suggestedPrice Float?
  createdById String
  createdAt   DateTime @default(now())
  sales       Sale[]
  movements   StockMovement[]

  @@index([accountId])
  @@index([accountId, categoryId])
}

model StockMovement {
  id          String    @id @default(cuid())
  productId   String
  product     Product @relation(fields: [productId], references: [id])
  quantity    Int
  costAtTime  Float?
  addedById   String
  createdAt   DateTime @default(now())

  @@index([productId])
}
```

دليل شرح قاعدة البيانات - Inventory & Sales SaaS

```
model Expense {
  id          String    @id @default(cuid())
  accountId   String
  account     Account   @relation(fields: [accountId], references: [id])
  description  String
  amount      Float
  date        DateTime @default(now())

  @@index([accountId])
}

model Sale {
  id          String    @id @default(cuid())
  accountId   String
  account     Account   @relation(fields: [accountId], references: [id])
  productId   String
  product     Product   @relation(fields: [productId], references: [id])
  categoryId  String
  quantity    Int
  soldPrice   Float
  costAtSale  Float
  soldById    String
  createdAt   DateTime @default(now())

  @@index([accountId, createdAt])
  @@index([accountId, categoryId, createdAt])
  @@index([accountId, productId, createdAt])
}

model DailyReport {
  id          String    @id @default(cuid())
  accountId   String
  account     Account   @relation(fields: [accountId], references: [id])
  date        DateTime
  totalSales  Float
  totalProfit Float
  totalUnitsSold Int
  stockValue  Float
  createdAt   DateTime @default(now())

  @@unique([accountId, date])
  @@index([accountId, date])
}
```

1. Account - المحل / الحساب الرئيسي

```
model Account {
  id          String          @id @default(cuid())
  name        String
  isActive    Boolean         @default(true)
  users       User[]
  categories  Category[]
  products    Product[]
  expenses    Expense[]
  sales       Sale[]
  reports     DailyReport[]
  createdAt   DateTime        @default(now())
}
```

Account هو الجذر الأساسي للنظام. كل محل أو Business يحصل على Account مستقل، وكل بياناته ترجع لهذا الحساب.

الحقول

- `id`: المعرّف الفريد للحساب. `id@` تعني Primary Key و `@default(cuid())` تجعل Prisma يولد القيمة تلقائياً.
- `name`: اسم المحل أو الشركة، مثل Massoud Fashion.
- `isActive`: يحدد إذا الحساب مسموح له باستخدام النظام. القيمة الافتراضية true. إيقافه أفضل من حذف بياناته.
- `users`: جميع المستخدمين المرتبطين بالمحل. العلاقة One-to-Many.
- `categories`: جميع فئات المنتجات التابعة للمحل.
- `products`: جميع المنتجات الخاصة بالمحل.
- `expenses`: جميع مصاريف المحل.
- `sales`: جميع عمليات البيع التابعة للمحل.
- `reports`: جميع DailyReport الخاصة بالحساب.
- `createdAt`: وقت إنشاء الحساب تلقائياً.

الدور في النظام

هو أساس الـ Multi-Tenant. وجود accountId على البيانات يسمح بفصل بيانات كل متجر عن الآخر، بحيث لا يرى محل A بيانات محل B.

2. User و UserRole - المستخدم والصلاحيات

```
enum UserRole {  
  SUPER_ADMIN  
  OWNER  
  WAREHOUSE  
}  
  
model User {  
  id      String    @id @default(cuid())  
  email   String    @unique  
  password String  
  role    UserRole  
  accountId String?  
  account Account? @relation(fields: [accountId], references: [id])  
  createdAt DateTime @default(now())  
}
```

User يمثل أي شخص يستطيع تسجيل الدخول. UserRole يحدد نوع المستخدم وما هي صلاحياته.

الأدوار

- SUPER_ADMIN: مسؤول عن المنصة كاملة، وليس تابعا لمحل محدد. لذلك accountId يمكن أن يكون null.
- OWNER: صاحب المحل. يكون مرتبطا بـ Account ويملك صلاحيات الإدارة والمال والتقارير حسب تصميم النظام.
- WAREHOUSE: موظف المخزن أو المستودع، ويتبع لنفس Account الخاص بالمحل.

الحقول

- id: ID فريد للمستخدم.
- email: بريد المستخدم. @unique يمنع تسجيل نفس البريد مرتين.
- password: كلمة المرور المخزنة. يجب ألا تكون Plain Text؛ يجب تخزين Hash آمن مثل Argon2 أو bcrypt.
- role: واحدة فقط من القيم المحددة في UserRole.
- accountId: يحدد المحل الذي ينتمي إليه المستخدم. عليه `?` لأنه اختياري لـ SUPER_ADMIN.
- account: تعريف علاقة Prisma بين User.accountId و Account.id.
- createdAt: وقت إنشاء المستخدم.

قاعدة مهمة

لا تعتمد على role في الواجهة الأمامية فقط. الصلاحيات الحساسة يجب التحقق منها في Backend/Database أيضا.

3. Category - تصنيف المنتجات

```
model Category {
  id          String    @id @default(cuid())
  accountId   String
  account     Account   @relation(fields: [accountId], references: [id])
  name        String
  products    Product[]
  createdAt   DateTime  @default(now())

  @@index([accountId])
}
```

Category ينظم المنتجات داخل المحل، مثل Pantalon و Chemise و Shoes. نفس اسم الفئة يمكن أن يوجد في أكثر من محل لأن كل Category مرتبطة بـ accountId مختلف.

الحقول

- id: ID `الفئة.
- accountId: المحل الذي تملك هذه الفئة.
- account: العلاقة مع Account.
- name: اسم الفئة.
- products: كل المنتجات الموجودة تحت هذه الفئة.
- createdAt: تاريخ إنشاء الفئة.
- index([accountId]): Index@@ لتسريع الاستعلامات التي تجلب فئات محل معين.

الدور في النظام

يسهل الفلترة، تنظيم المخزون، وتقارير المبيعات حسب الفئة.

4. Product - الحالة الحالية للمنتج

```
model Product {
  id          String          @id @default(cuid())
  accountId   String
  account     Account         @relation(fields: [accountId], references: [id])
  categoryId  String
  category    Category        @relation(fields: [categoryId], references: [id])
  name        String
  imageUrl    String
  color       String
  wholesaleCost Float?
  profitMargin Float?
  currentStock Int            @default(0)
  suggestedPrice Float?
  createdById String
  createdAt   DateTime        @default(now())
  sales        Sale[]
  movements    StockMovement[]

  @@index([accountId])
  @@index([accountId, categoryId])
}
```

Product يحفظ الوضع الحالي للمنتج: اسمه، فئته الحالية، كلفته الحالية، كمية المخزون الحالية، هامش الربح، والسعر المقترح.

الحقول

- `id``: ID` المنتج.
- `accountId``: المحل الذي يملك المنتج.
- `categoryId``: الفئة التي ينتمي إليها المنتج حالياً.
- `account`` و `category``: علاقات Prisma مع Account و Category.
- `name``: اسم المنتج.
- `imageUrl``: رابط صورة المنتج. تخزين الرابط أفضل من تخزين ملف الصورة نفسه داخل Postgres.
- `color``: لون المنتج.
- `wholesaleCost``: كلفة شراء القطعة حالياً. Optional حتى يمكن إنشاء المنتج قبل أن يضع Owner الكلفة.
- `profitMargin``: هامش الربح المستهدف. Optional لنفس فكرة مرونة الإدخال.
- `currentStock``: عدد القطع الموجودة حالياً. يبدأ من صفر.
- `suggestedPrice``: السعر الذي يقترحه النظام بناء على الكلفة والهامش والمصاريف حسب طريقة الحساب.
- `createdById``: ID` المستخدم الذي أنشأ المنتج؛ مفيد للـ Audit.

- `sales`: جميع مبيعات المنتج.
- `movements`: جميع حركات المخزون للمنتج.
- `index([accountId])@@`: يسرع جلب منتجات محل محدد.
- `index([accountId, categoryId])@@`: يسرع جلب منتجات فئة محددة داخل محل.

مهم

Product هو Current State. إذا wholesaleCost تغيرت اليوم، هذا لا يجب أن يغيّر كلفة المبيعات القديمة. التاريخ الحقيقي محفوظ في Sale و StockMovement.

5. StockMovement - سجل تاريخ المخزون

```
model StockMovement {
  id          String   @id @default(cuid())
  productId   String
  product     Product  @relation(fields: [productId], references: [id])
  quantity    Int
  costAtTime  Float?
  addedById   String
  createdAt   DateTime @default(now())

  @@index([productId])
}
```

StockMovement هو سجل تاريخي لكل حركة مخزون مسجلة. الهدف ألا نفقد تاريخ الشحنات والكلف حتى لو تغيرت بيانات Product لاحقاً.

مثال

```
January -> +50 Jeans, cost $10
February -> +30 Jeans, cost $12
March -> +100 Jeans, cost $11
```

الحقول

- `id`: ID حركة المخزون.
- `productId`: المنتج الذي تتعلق به الحركة.
- `product`: العلاقة مع Product.
- `quantity`: الكمية المسجلة في الحركة.

- `costAtTime`: كلفة القطعة وقت حدوث هذه الحركة. هذه القيمة تحفظ التاريخ ولا تتغير عندما تتغير `wholesaleCost` الحالية.
- `addedById`: المستخدم الذي سجل الحركة.
- `createdAt`: متى سجلت الحركة.
- `index([productId])@@`: يسرع قراءة تاريخ المخزون لمنتج محدد.

الدور في النظام

Product.currentStock يجب: كم قطعة عندي الآن؟ أما StockMovement فيجب: كيف وصل المخزون إلى هذه الكمية؟

6. Expense - مصاريف المحل

```
model Expense {
  id          String    @id @default(cuid())
  accountId   String
  account     Account   @relation(fields: [accountId], references: [id])
  description String
  amount      Float
  date        DateTime  @default(now())
  @@index([accountId])
}
```

Expense يسجل المصاريف العامة للمحل، مثل الإيجار والكهرباء والإنترنت والنقل والرواتب، حسب ما يسمح به النظام.

الحقول

- `ID`: id المصروف.
- `accountId`: المحل الذي دفع المصروف.
- `account`: العلاقة مع Account.
- `description`: وصف المصروف، مثل Electricity أو Monthly Rent.
- `amount`: قيمة المصروف.
- `date`: تاريخ المصروف، مهم لتقارير اليوم والشهر والسنة.
- `index([accountId])@@`: يسرع جلب مصاريف محل محدد.

لماذا هو مهم؟

لأن الربح الحقيقي لا يساوي فقط سعر البيع ناقص كلفة البضاعة. المصاريف التشغيلية تؤثر على الربح وعلى طريقة التسعير المقترح.

7. Sale - المصدر الحقيقي لكل عملية بيع

```
model Sale {
  id          String   @id @default(cuid())
  accountId   String
  account     Account  @relation(fields: [accountId], references: [id])
  productId   String
  product     Product  @relation(fields: [productId], references: [id])
  categoryId  String
  quantity    Int
  soldPrice   Float
  costAtSale  Float
  soldById    String
  createdAt   DateTime @default(now())

  @@index([accountId, createdAt])
  @@index([accountId, categoryId, createdAt])
  @@index([accountId, productId, createdAt])
}
```

Sale هو أهم سجل تاريخي للمبيعات. كل عملية بيع تنشئ Row جديدا يحتوي على ما حدث فعليا لحظة البيع.

الحقول

- `id``: ID `عملية البيع.
- `accountId``: المحل الذي حدثت فيه عملية البيع.
- `productId``: المنتج الذي بيع.
- `product``: علاقة Prisma مع Product.
- `categoryId``: Snapshot للفئة وقت البيع، حتى لو تغير تصنيف Product لاحقا تبقى التقارير القديمة صحيحة.
- `quantity``: عدد القطع المباعة.
- `soldPrice``: السعر الفعلي الذي بيعت فيه القطعة، وقد يختلف عن `suggestedPrice`.
- `costAtSale``: Snapshot لكلفة القطعة وقت البيع؛ لا يتغير عندما تتغير `wholesaleCost` لاحقا.
- `soldById``: المستخدم الذي نفذ البيع.
- `createdAt``: تاريخ ووقت عملية البيع.

لماذا Snapshot داخل Sale؟

مثال: إذا كانت كلفة Jeans وقت البيع \$10 ثم أصبحت لاحقا \$14، لا يجوز أن يعاد احتساب ربح البيع القديم على كلفة \$14. لذلك نحفظ $\text{costAtSale} = 10$ داخل Sale نفسه.

```
At sale time:
  soldPrice = $17
  costAtSale = $10
  profit    = $7

Later product cost becomes $14
Historical sale still keeps costAtSale = $10
```

ونفس المبدأ ينطبق على categoryId: إذا تغيرت فئة المنتج لاحقا، المبيعات القديمة تبقى محسوبة ضمن الفئة التي كانت صحيحة وقت البيع.

Indexes وظيفة الـ

- `index([accountId, createdAt])@@``: مناسب لتقارير مبيعات محل خلال فترة زمنية.
- `index([accountId, categoryId, createdAt])@@@``: مناسب لتقارير المبيعات حسب الفئة والفترة.
- `index([accountId, productId, createdAt])@@@``: مناسب لتاريخ مبيعات منتج محدد ضمن فترة.

قاعدة أساسية

Sale هو Source of Truth للمبيعات. إذا اختلف DailyReport عن Sale، نثق بـ Sale ثم نعيد حساب التقرير.

8. DailyReport - ملخص يومي محسوب مسبقا

```
model DailyReport {
  id          String @id @default(cuid())
  accountId   String
  account     Account @relation(fields: [accountId], references: [id])
  date        DateTime
  totalSales  Float
  totalProfit Float
  totalUnitsSold Int
  stockValue  Float
  createdAt   DateTime @default(now())

  @@unique([accountId, date])
  @@index([accountId, date])
}
```

DailyReport ليس مصدرا أصليا للمبيعات. هو Cache أو ملخص جاهز محسوب من Sale وبيانات أخرى حتى يكون Dashboard سريع.

مثال

```
Date: 2026-09-04
Total Sales      = $4,200
Total Profit     = $1,350
Total Units Sold = 230
Stock Value      = $19,000
```

الحقول

- `ID`: id التقرير.
- `accountId`: المحل الذي يخصه التقرير.
- `date`: اليوم الذي يمثلته التقرير.
- `totalSales`: مجموع قيمة المبيعات لذلك اليوم.
- `totalProfit`: مجموع الربح المحسوب. إذا soldPrice سعر القطعة، فالمبدأ هو مجموع (soldPrice - costAtSale) × quantity.
- `totalUnitsSold`: مجموع الكميات المباعة.
- `stockValue`: قيمة المخزون حسب المنطق الذي تعتمد عليه عند إنشاء التقرير.
- `createdAt`: متى تم إنشاء التقرير.
- `unique([accountId, date])@@`: يمنع وجود أكثر من تقرير لنفس الحساب ونفس قيمة date.
- `index([accountId, date])@@`: يسرع قراءة تقرير حساب حسب التاريخ.

الدور في النظام

بدل أن يحسب Dashboard آلاف عمليات Sale كل مرة يفتح فيها المستخدم الصفحة، يمكن قراءة DailyReport الجاهز. وإذا حصل خطأ في التقرير، يمكن إعادة بنائه من البيانات الأصلية.

مثال كامل: كيف تتحرك البيانات بين الجداول؟

نفترض أن لدينا محل اسمه Massoud Fashion.

1. إنشاء **Account: Massoud Fashion** يحصل على Account خاص به.

2. إنشاء **Users: Omar** يكون OWNER و Ali يكون WAREHOUSE، وكلاهما مرتبط بنفس accountId.

3. إنشاء **Category**: يتم إنشاء Category باسم Pantalon.

4. إنشاء **Product**: يتم إنشاء Black Jeans داخل Pantalon، ويبدأ currentStock من 0.

5. دخول شحنة: StockMovement يسجل +100 قطعة بكلفة \$10 للقطعة، ويتم تحديث Product.currentStock إلى 100.

6. تحديد التسعير: Owner يضع $\text{wholesaleCost} = 10\$$ و profitMargin، فيحسب النظام suggestedPrice حسب قواعده.

7. عملية بيع: يباع قطعتان بسعر فعلي \$15 للقطعة. ينشأ Sale مع $\text{quantity} = 2$ و $\text{soldPrice} = 15$ و $\text{costAtSale} = 10$ و categoryId الحالية.

8. تحديث المخزون: currentStock يصبح 98.

9. تغير الكلفة لاحقاً: إذا أصبحت $\text{wholesaleCost} = 12\$$ ، يبقى Sale القديم محتفظاً بـ $\text{costAtSale} = 10\$$.

10. إنشاء تقرير يومي: Job يحسب totalSales و totalProfit و totalUnitsSold ويحفظها في DailyReport.

```
100 stock
- 2 sold
= 98 currentStock

Sale total = 2 x $15 = $30
Gross profit = 2 x ($15 - $10) = $10
```

تقسيم الجداول حسب وظيفتها

A. Current State - الحالة الحالية

• Account

• User

• Category

• Product

هذه الجداول تخبرك كيف يبدو النظام الآن. مثال: $\text{Product.currentStock} = 98$.

B. Historical Records - السجل التاريخي

• StockMovement

• Expense

• Sale

هذه الجداول تخبرك ماذا حدث عبر الزمن. وهي مهمة للتقارير، التدقيق، وعدم تغيير الماضي عندما تتغير القيم الحالية.

C. Calculated Cache - بيانات محسوبة مسبقا

• DailyReport

هذا الجدول يختصر النتائج الجاهزة لتسريع القراءة. لا يجب اعتباره بديلا عن البيانات الأصلية.

الخلاصة العملية

Account = لمن هذه البيانات؟ User = من يعمل عليها؟ Product = ما وضع المنتج الآن؟ StockMovement = كيف تحرك المخزون؟ Sale = ماذا بيع فعليا؟ Expense = ماذا دفع المحل؟ DailyReport = ما نتيجة اليوم؟

ملاحظات تقنية مهمة قبل التنفيذ النهائي

1. استخدم **Decimal** للأموال بدل **Float**

Float مناسب للحسابات التقريبية، لكنه قد يسبب مشاكل precision بالأموال. في Prisma/PostgreSQL يفضل

استخدام Decimal للحقول المالية.

```
// Better for money
wholesaleCost Decimal?
suggestedPrice Decimal?
amount         Decimal
soldPrice      Decimal
costAtSale     Decimal
totalSales     Decimal
totalProfit    Decimal
stockValue     Decimal
```

2. createdById / addedById / soldById

في الـ Schema الحالية هذه الحقول String فقط. الأفضل عند اكتمال التصميم ربطها بعلاقة Foreign Key إلى User حتى

تمنع IDs غير موجودة وتحصل على Audit أقوى.

3. تأكد أن **Category** و **Product** من نفس **Account**

وجود accountId في Product و Category مهم، لكن Backend يجب أن يمنع ربط Product من Account A بـ

Category من Account B.

4. انتبه إلى **DailyReport.date**

DateTime يحتوي على الوقت أيضا. إذا كنت تقصد يوما واحدا، يجب توحيد قيمة الوقت أو اعتماد منطق تاريخ واضح حتى لا تنشأ قيمتان مختلفتان لنفس اليوم.

ملخص أخير

Table	الدور المختصر
Account	المحل والجذر الرئيسي لكل البيانات.
User	المستخدم، دوره، والحساب الذي ينتمي إليه.
Category	تنظيم المنتجات ضمن فئات.
Product	الحالة الحالية للمنتج والمخزون والتسعير.
StockMovement	التاريخ الكامل لحركات المخزون والكلفة وقت الحركة.
Expense	المصاريف التشغيلية والتاريخ المالي للمحل.
Sale	المصدر الحقيقي لكل بيع مع snapshots تاريخية ثابتة.
DailyReport	ملخص يومي سريع محسوب من البيانات الأصلية.